🔵 Git Intermediate Interview Mastery — I1 to I8

> **How to use this guide:** Every topic follows: concept → internals → short interview answer → deep dive → real-world example → interview Q&A → gotchas → connections. ⚠ marks high-frequency interview traps. Master these and you cover 80% of what senior DevOps/SRE interviews actually test on Git.

I1. Git Internals ⚠

> This is the topic that separates candidates who *use* Git from candidates who *understand* Git. Interviewers at senior/staff level will probe here.

The Four Object Types: Blob, Tree, Commit, Tag

What it is (Simple Terms)

Git is fundamentally a **content-addressable key-value store**. Everything Git tracks — file contents, directory structures, commits, tags — is stored as one of four object types inside `.git/objects/`. Every object is identified by its SHA-1 hash.

The Four Object Types

---

1. Blob (Binary Large Object)

A blob stores **raw file content** — nothing else. No filename, no permissions, no path. Just bytes.

```
# Manually create a blob object
echo "Hello, Git" | git hash-object --stdin -w
# Output: 8ab686eafeb1f44702738c8b0f24f2567c36da6d

# Inspect it
git cat-file -t 8ab686e    # type: blob
git cat-file -p 8ab686e    # content: Hello, Git

# See blob for a tracked file
git ls-files -s src/auth.js
# 100644 a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c 0  src/auth.js
#  mode   SHA-1 of blob                   stage  path

git cat-file -p a3f9d2c    # prints file contents
```

Key insight: Two files with identical content share **one blob object**. Git deduplicates at the content level automatically.

---

2. Tree

A tree stores a **directory listing** — it maps filenames and permissions to blob SHAs (for files) or other tree SHAs (for subdirectories). It is the directory snapshot at a point in time.

```
# Inspect a tree
git cat-file -p HEAD^{tree}
# Output:
# 100644 blob a3f9d2c  .gitignore
# 100644 blob b7e1234  README.md
# 040000 tree c4d5678  src
# 040000 tree d5e6789  tests

# Inspect a subtree (the src/ directory)
git cat-file -p c4d5678
# 100644 blob e6f7890  auth.js
# 100644 blob f7a8901  index.js
```

Mode values:

```
100644  → regular file
100755  → executable file
120000  → symbolic link
040000  → directory (subtree)
160000  → gitlink (submodule)
```

---

3. Commit

A commit object stores:

- Pointer to the **root tree** (the full directory snapshot)
- Pointer to **parent commit(s)** (zero for initial, one for regular, two+ for merge commits)
- Author name, email, timestamp
- Committer name, email, timestamp (different from author for rebased commits)
- Commit message

```
# Inspect a commit
git cat-file -p HEAD
# Output:
# tree   9a8b765d4e3f2c1b0a9f8e7d6c5b4a3f2e1d0c9b
# parent 1f2e3d4c5b6a7f8e9d0c1b2a3f4e5d6c7b8a9f0
# author  Jane Smith <jane@company.com> 1700000000 +0530
# committer Jane Smith <jane@company.com> 1700000000 +0530
#
# feat: add OAuth2 authentication
#
```

```
# Implements Google and GitHub providers.
# Closes #142
```

Key insight: The commit does NOT store the diff. It stores a full snapshot (via the tree pointer). Git computes diffs on the fly by comparing trees.

---

4. Tag Object (Annotated Tag)

An annotated tag is a full Git object that stores:

- Pointer to the tagged object (usually a commit)
- Tagger name, email, timestamp
- Tag message
- Optional GPG signature

```
# Inspect an annotated tag
git cat-file -p v2.1.0
# Output:
# object a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d
# type   commit
# tag    v2.1.0
# tagger Jane Smith <jane@company.com> 1700000000 +0530
#
# Release 2.1.0 - OAuth2 integration
```

Lightweight tags are NOT objects — they are just refs (files containing a SHA).

The Object Model — Full Picture

```
Commit Object
│
├── tree SHA ──────► Tree Object (root /)
│                    ├── blob SHA ──► Blob (README.md content)
│                    ├── blob SHA ──► Blob (.gitignore content)
│                    └── tree SHA ──► Tree Object (src/)
│                                     ├── blob SHA ──► Blob (auth.js)
│                                     └── blob SHA ──► Blob (index.js)
│
└── parent SHA ────► Previous Commit Object
                     └── tree SHA ──► (previous snapshot)
```

How SHA-1 Hashing Works — Content-Addressable Storage

What it is

Git computes a SHA-1 hash from the **type + size + content** of every object and uses that hash as the object's name. This is called **content-addressable storage** — the address IS the content.

The Hash Formula

```
SHA1("blob " + content_length + "\0" + content)
SHA1("tree " + content_length + "\0" + content)
SHA1("commit " + content_length + "\0" + content)
```

```
# Reproduce Git's hash manually
echo -n "blob 11\0Hello, Git" | sha1sum
# Roughly equivalent to what git hash-object computes

# Verify yourself
printf "blob 11\0Hello, Git" | sha1sum
# 8ab686eafeb1f44702738c8b0f24f2567c36da6d
```

Why This Architecture Matters

1. Data integrity across distributed clones Every byte of history is verified. If any object is corrupted in transit, the hash won't match and Git rejects it.

2. Deduplication If you have 100 branches that all share a `README.md` with the same content, they all point to the **same blob object**. Stored exactly once.

3. Immutability You can never change a commit. If you amend or rebase, you get a **new object with a new hash**. The old object still exists until garbage collected.

4. Caching If the hash matches, the content matches — 100% guaranteed. Git uses this for efficient packfile delta compression.

Storage Location

```
# Object stored at .git/objects/ab/cd1234...
# First 2 chars = directory, rest = filename
ls .git/objects/8a/
# b686eafeb1f44702738c8b0f24f2567c36da6d
```

Short Interview Answer

> "Git has four object types: blob stores file content, tree stores directory structure, commit stores a snapshot pointer with metadata and parent links, and annotated tag is a full object wrapping a commit. All are stored in `.git/objects/` identified by their SHA-1 hash — the hash is computed from the content itself, making Git a content-addressable store. This gives Git data integrity, automatic deduplication, and immutability by design."

The `.git` Directory Structure in Full

```
.git/
│
├── HEAD                    # Current branch pointer: "ref:
refs/heads/main"
├── ORIG_HEAD               # Saved HEAD before dangerous operations
(merge, reset)
├── MERGE_HEAD              # SHA of commit being merged (during a merge)
├── FETCH_HEAD              # Last fetched SHA from remote
├── CHERRY_PICK_HEAD        # SHA being cherry-picked (during cherry-pick)
├── REBASE_HEAD             # SHA being rebased (during rebase)
│
├── config                  # Repo-level config (remotes, tracking,
settings)
├── description             # Used by GitWeb (not relevant for daily use)
├── index                   # Binary file = the staging area / index
│
├── objects/                # All Git objects (content-addressable store)
│   ├── ab/                 # First 2 hex chars of SHA
│   │   └── cd1234...       # Remaining 38 chars = filename
│   ├── info/               # Object info for dumb HTTP transport
│   └── pack/               # Packed objects (packfiles)
│       ├── pack-abc.idx    # Pack index (fast lookup)
│       └── pack-abc.pack   # Pack file (delta-compressed objects)
│
├── refs/                   # References (pointers to SHAs)
│   ├── heads/              # Local branch refs
│   │   ├── main            # Contains SHA of tip of main
│   │   └── feature/login   # Contains SHA of tip of feature/login
│   ├── remotes/            # Remote-tracking refs
│   │   └── origin/
│   │       ├── main        # Last known state of origin/main
│   │       └── HEAD        # Default branch of origin
│   └── tags/               # Tag refs
│       └── v2.1.0          # Lightweight: SHA of commit; Annotated: SHA
of tag object
│
├── logs/                   # Reflogs — history of where refs pointed
│   ├── HEAD                # Every HEAD movement ever
│   └── refs/
│       └── heads/
│           └── main        # Every time main moved
│
├── hooks/                  # Hook scripts (pre-commit, post-commit, etc.)
│   ├── pre-commit.sample
│   └── commit-msg.sample
│
├── info/
│   └── exclude             # Local .gitignore (not committed)
```

```
│
└── modules/                # Submodule repos (when using git submodule)
```

```
# Explore it yourself
cat .git/HEAD
# ref: refs/heads/main

cat .git/refs/heads/main
# a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d

cat .git/config
# [core]
#     repositoryformatversion = 0
#     filemode = true
#     bare = false
# [remote "origin"]
#     url = git@github.com:org/repo.git
#     fetch = +refs/heads/*:refs/remotes/origin/*
# [branch "main"]
#     remote = origin
#     merge = refs/heads/main
```

Packfiles, Loose Objects, and `git gc`

Loose Objects

When you commit, Git initially stores objects as **loose objects** — individual files in `.git/objects/xx/yyyy...`. This is fast for writing but inefficient for storage and reading large histories.

```
# Count loose objects
git count-objects
# 47 objects, 382 kilobytes

# Verbose count
git count-objects -v
# count: 47          ← loose objects
# size: 382          ← size in KB
# in-pack: 2891      ← objects in packfiles
# packs: 1           ← number of packfiles
# size-pack: 4820    ← size of packfiles in KB
# prune-packable: 0
# garbage: 0
```

Packfiles

When there are many loose objects (typically after `git push` or `git gc`), Git **packs** them:

1. Collects related objects
2. Finds similar objects and stores them as **deltas** (differences)
3. Writes a `.pack` file (packed objects) and `.idx` file (index for fast lookup)

```
# Manually pack objects
git pack-objects .git/objects/pack/pack

# Run full garbage collection (packs loose objects, prunes unreachable)
git gc

# Aggressive GC (deeper delta compression, slower)
git gc --aggressive

# Prune unreachable objects older than 2 weeks
git prune --expire=2.weeks.ago
```

`git gc` — What It Does

```
git gc:
  1. Packs loose objects into packfiles
  2. Removes redundant loose objects already in packs
  3. Prunes unreachable objects (orphaned commits from abandoned rebases,
etc.)
  4. Repacks existing packfiles for efficiency
  5. Updates the commit graph for faster log traversal
```

Git runs `git gc --auto` automatically after operations like `git fetch` and `git push` when loose object count exceeds a threshold (default: 6700 objects).

⚠ Gotchas

- Objects referenced by reflog are NOT pruned by `git gc` even if unreachable from any branch. Reflog retains them for 90 days by default.
- Force-pruning everything: `git gc --prune=now` — dangerous, can delete objects reflog still references.
- On large repos, `git gc --aggressive` can take hours. GitHub/GitLab run this server-side on their schedules.

Refs, HEAD, ORIG_HEAD, FETCH_HEAD, MERGE_HEAD

HEAD

The pointer to your current position in the repo.

```
cat .git/HEAD
# ref: refs/heads/main      ← attached (normal state)
```

```
# a3f9d2c4e7b18f...         ← detached HEAD (points directly to SHA)
```

ORIG_HEAD

Saved automatically before operations that significantly move HEAD — `git merge`, `git rebase`, `git reset`. Provides an **undo point**.

```
# After a bad merge or reset:
git reset --hard ORIG_HEAD    # go back to where you were before
```

FETCH_HEAD

Records what was fetched in the last `git fetch`. Contains the SHA and branch info for each fetched ref.

```
cat .git/FETCH_HEAD
# a3f9d2c4... branch 'main' of github.com:org/repo

# Merge what was just fetched (what pull does internally)
git fetch origin main
git merge FETCH_HEAD
# equivalent to: git pull origin main
```

MERGE_HEAD

Created when a merge is in progress. Contains the SHA of the commit being merged in.

```
# Only exists during an in-progress merge
cat .git/MERGE_HEAD
# b7e12340f5a8b9c...
```

CHERRY_PICK_HEAD / REBASE_HEAD

Similar — exist only during in-progress cherry-pick or rebase operations.

```
# Relative ref syntax using these
git show ORIG_HEAD        # what HEAD was before the last dangerous op
git diff ORIG_HEAD HEAD   # what changed since the dangerous op
```

Ref Resolution Order

When you type a ref name like `main`, Git resolves it in this order:

```
1. .git/main
2. .git/refs/main
3. .git/refs/tags/main
4. .git/refs/heads/main       ← most common
5. .git/refs/remotes/main
6. .git/refs/remotes/main/HEAD
```

How `git log` Traverses the DAG

The DAG — Directed Acyclic Graph

Git's commit history is a **DAG** — each commit points to its parent(s), forming a directed graph with no cycles. `git log` is a **graph traversal algorithm** that walks this DAG.

```
C5 ← C4 ← C3 ← C2 ← C1   (main)
           ↑
           F3 ← F2 ← F1   (feature, branched from C3)
```

How `git log` Traversal Works

```
1. Start at HEAD (or specified commit)
2. Add it to a priority queue (sorted by commit date by default)
3. Print the commit
4. Add its parent(s) to the queue
5. Repeat until queue is empty or limit reached
```

For merge commits (two parents), both parents are enqueued — this is how `git log` covers the full history.

```
# Limit graph traversal
git log --ancestry-path C3..C5   # only commits on the direct path

# Exclude commits reachable from a ref
git log main..feature            # commits in feature not in main
git log main...feature           # commits in either but not both
(symmetric diff)

# Speed up traversal on large repos
git log --first-parent           # only follow first parent (skip merge
branches)
git log --simplify-by-decoration # only show commits with refs (tags,
branch tips)
```

Commit Graph File (Performance Optimization)

Git 2.18+ stores a pre-computed commit graph file:

```
git commit-graph write           # pre-compute the graph
git config core.commitGraph true # use it automatically
```

This speeds up `git log`, `git merge-base`, and `git blame` dramatically on large repos.

Short Interview Answer (Full l1)

> "Git stores all data as four object types in `.git/objects/`: blobs for file content, trees for directory structure, commits for snapshots with parent links, and annotated tags as full objects. Every object is SHA-1 hashed from its content — this is content-addressable storage, giving Git data integrity and deduplication. The commit history forms a DAG that git log traverses. Objects start as loose files and get packed into packfiles by git gc for efficiency. Special refs like ORIG_HEAD, FETCH_HEAD, and MERGE_HEAD are safety pointers Git writes during complex operations."

Common Interview Questions

Q: What is stored in a Git commit object?

> A commit stores a pointer to a root tree (the directory snapshot), pointer(s) to parent commits, author info, committer info, and the commit message. It does NOT store diffs — diffs are computed by comparing two tree objects.

Q: If two files have identical content, how does Git store them?

> Git stores only one blob object. Both files' tree entries point to the same blob SHA. This is automatic deduplication via content-addressable storage.

Q: What is the difference between author and committer in a commit?

> Author is who wrote the change. Committer is who applied it to the repository. They differ when you cherry-pick or rebase someone else's commit — the original author is preserved, but you become the committer.

Q: What does `git gc` do and when should you run it?

> `git gc` packs loose objects into delta-compressed packfiles, removes redundant loose objects, and prunes unreachable objects. Git runs it automatically via `git gc --auto` but you'd run it manually on large repos to reclaim space or after bulk operations like filter-repo.

⚠ Gotchas

- The `index` file (`.git/index`) is the staging area. It is binary — don't edit it manually.
- ORIG_HEAD is overwritten every time you do a merge/reset. If you do two dangerous operations, you lose the ORIG_HEAD from the first.
- Packfiles cannot be read directly — you need `git cat-file` or `git unpack-objects` to inspect them.

- On a fresh `git init`, `.git/objects/` has `info/` and `pack/` subdirectories but no objects. The first commit creates the first objects.

I2. Rewriting History ⚠

> The most dangerous and most powerful set of Git commands. Interviewers test both the mechanics AND the judgment of when to use them.

`git rebase` — What It Is, How It Replays Commits

What it is (Simple Terms)

Rebase **moves or replays** a sequence of commits onto a new base commit. It rewrites history to make it appear as if your branch was created from a different point.

The Problem it Solves

You branch off `main` at commit C2. While you work, `main` gets new commits C3 and C4. Your feature branch is now "behind" main. Merging creates a merge commit. Rebasing rewrites your commits so they appear to have been made after C4 — clean linear history, no merge commit.

How it Works Internally — Step by Step

```
Before rebase:
  main:    C1 → C2 → C3 → C4
                ↑
  feature:      F1 → F2 → F3

Command: git checkout feature && git rebase main

Git internally:
  1. Finds common ancestor of feature and main: C2
  2. Saves the diff (patch) for F1, F2, F3
  3. Moves feature pointer to tip of main (C4)
  4. Re-applies F1's patch → creates F1' (new SHA)
  5. Re-applies F2's patch → creates F2' (new SHA)
  6. Re-applies F3's patch → creates F3' (new SHA)
  7. Points feature to F3'

After rebase:
  main:    C1 → C2 → C3 → C4
                            ↖
  feature:                  F1' → F2' → F3'

F1, F2, F3 still exist as orphaned objects until git gc
F1', F2', F3' are NEW commits with NEW SHAs
```

```
# Basic rebase
git checkout feature/login
git rebase main

# Rebase onto a specific commit
git rebase abc1234

# Rebase a branch without checking it out
git rebase main feature/login

# Abort a rebase in progress
git rebase --abort

# Continue after resolving a conflict
git rebase --continue

# Skip a commit that has conflicts (the patch becomes empty)
git rebase --skip
```

Interactive Rebase (`rebase -i`) — Squash, Fixup, Reword, Drop, Edit

What it is

Interactive rebase lets you **edit the list of commits** before replaying them. You can reorder, combine, rename, split, or delete commits entirely.

```
# Interactive rebase for last 5 commits
git rebase -i HEAD~5

# Interactive rebase from a specific commit (exclusive — that commit not
included)
git rebase -i abc1234

# Interactive rebase onto main (all commits since branching from main)
git rebase -i $(git merge-base HEAD main)
```

The Interactive Rebase Editor

```
pick a3f9d2c feat: add login form
pick b7e1234 fix: typo in login form
pick c4d5678 fix: another typo
pick d5e6789 feat: add OAuth button
pick e6f7890 WIP: oauth integration
pick f7a8901 feat: complete OAuth flow

# Commands:
# p, pick   = use commit as-is
```

```
# r, reword = use commit, but edit the commit message
# e, edit   = use commit, but pause for amending
# s, squash = meld into previous commit, keep both messages
# f, fixup  = meld into previous commit, discard this message
# d, drop   = remove commit entirely
# b, break  = pause here (continue with 'git rebase --continue')
# x, exec   = run shell command
```

Common Patterns

```
# Squash 3 fixes into the feat commit above them:
pick a3f9d2c feat: add login form
squash b7e1234 fix: typo in login form
squash c4d5678 fix: another typo
pick d5e6789 feat: add OAuth button
squash e6f7890 WIP: oauth integration
fixup f7a8901 feat: complete OAuth flow

# Result: 2 clean commits
# "feat: add login form" (with typo fixes absorbed)
# "feat: add OAuth button" (with WIP absorbed, message discarded)
```

```
# --autosquash: automatically organize fixup! and squash! commits
# If you commit with message "fixup! feat: add login form"
git commit -m "fixup! feat: add login form"
git rebase -i --autosquash HEAD~10
# Git automatically moves and marks it as fixup
```

Splitting a Commit

```
# In the rebase editor, mark the commit as 'edit'
edit a3f9d2c feat: add login form AND OAuth

# Rebase pauses at that commit
git reset HEAD^                    # unstage everything from that commit
git add src/login.js
git commit -m "feat: add login form"
git add src/oauth.js
git commit -m "feat: add OAuth button"
git rebase --continue
```

`git commit --amend` — When to Use and When Not To

```

```

```
# Fix the last commit message
git commit --amend -m "feat: add OAuth2 login with Google provider"

# Add a forgotten file to the last commit
git add forgotten-file.js
git commit --amend --no-edit     # --no-edit keeps the same message

# Change the author of the last commit
git commit --amend --author="Jane Smith <jane@company.com>"
```

When It's Safe vs Dangerous

Situation	Safe?
Commit is only local, not pushed	✅ Safe
Commit pushed to your personal feature branch, no one else has it	⚠ Be careful, force push needed
Commit pushed to shared branch (main, develop)	❌ DANGEROUS — rewrites shared history

---

⚠ `git reset` — Soft vs Mixed vs Hard

What it is

`git reset` moves the **current branch pointer** to a specified commit. The three modes control what happens to the staging area and working directory.

The Three Modes Visualized

```
Your repo state:
  Working Dir:  [modified files]
  Staging Area: [staged files]
  Repository:   HEAD → C3 → C2 → C1

git reset --soft HEAD~1:
  Working Dir:  [modified files — UNCHANGED]
  Staging Area: [staged files + C3's changes — UNCHANGED + C3 added]
  Repository:   HEAD → C2 → C1   (C3 undone, its changes still staged)

git reset --mixed HEAD~1:  (DEFAULT)
  Working Dir:  [modified files + C3's changes — in working dir]
  Staging Area: [empty — cleared]
  Repository:   HEAD → C2 → C1

git reset --hard HEAD~1:
  Working Dir:  [C3's changes GONE — working dir matches C2]
```

```
  Staging Area: [empty]
  Repository:   HEAD → C2 → C1
```

```
# Soft: undo commit, keep changes staged (want to re-commit differently)
git reset --soft HEAD~1

# Mixed (default): undo commit and staging, keep changes in working dir
git reset HEAD~1
git reset --mixed HEAD~1

# Hard: undo commit AND discard all changes ⚠ DATA LOSS
git reset --hard HEAD~1

# Reset to a specific commit
git reset --hard abc1234

# Reset a specific file in staging area (unstage it)
git reset HEAD file.txt
# Modern equivalent:
git restore --staged file.txt

# Undo a hard reset using reflog (recovery)
git reflog
git reset --hard HEAD@{2}
```

Quick Memory Aid

```
--soft  = move branch pointer only        → changes land in staging
--mixed = move branch + clear staging     → changes land in working dir
--hard  = move branch + clear everything  → changes are GONE
```

`git revert` vs `git reset` — The Critical Difference

The Fundamental Difference

	`git reset`	`git revert`
What it does	Moves branch pointer backward	Creates a NEW commit that undoes changes
Rewrites history	Yes	No
Safe for shared branches	No	**Yes**
Audit trail	No — commit disappears	Yes — revert commit recorded

	git reset	git revert
Can undo multiple commits	Yes	Yes (one revert per commit)
Recoverable	Via reflog	Always

```
# Revert the last commit (creates a new "undo" commit)
git revert HEAD

# Revert a specific commit
git revert abc1234

# Revert without immediately committing (stage the revert)
git revert --no-commit abc1234

# Revert a range of commits
git revert HEAD~3..HEAD

# Revert a merge commit (must specify which parent to revert to)
git revert -m 1 <merge-commit-sha>
# -m 1 = revert to first parent (the branch you merged into)
```

When to Use Which

```
git reset → for LOCAL history cleanup before pushing
            → squashing WIP commits on your feature branch
            → undoing a commit you haven't shared with anyone

git revert → for SHARED branches (main, develop, release)
           → undoing a bad deploy in production
           → when you need an audit trail
           → CI/CD pipelines where "undo" must be visible
```

Real-World Scenario

```
Bad commit deployed to production: commit abc1234

Wrong approach:
  git reset --hard HEAD~1
  git push --force main
  → Destroys history, breaks teammates' repos, no audit trail

Right approach:
  git revert abc1234
  git push origin main
```

```
  → Creates revert commit, history preserved, CI reruns, audit trail
intact
```

⚠ Force Push and `--force-with-lease`

Why Force Push Exists

After rebasing or amending, your local branch has rewritten commits. Remote still has old commits. A normal `git push` is rejected because histories have diverged. Force push overwrites the remote.

```
# Force push ⚠ DANGEROUS
git push --force origin feature/login
# Silently overwrites ANYTHING on remote, even if teammates pushed

# Force push with lease ✅ SAFER
git push --force-with-lease origin feature/login
# Fails if remote has commits you haven't fetched
# "Lease" = you hold a lease on the remote state; push fails if state
changed

# Force push with lease specifying expected SHA
git push --force-with-lease=origin/feature/login:abc1234 origin
feature/login
```

The `--force-with-lease` Safety Net

```
Scenario: You and teammate both have feature/login checked out.
Teammate pushes a new commit to origin/feature/login.
You rebase and try to force push.

git push --force:
  → Silently overwrites teammate's commit. GONE.

git push --force-with-lease:
  → Detects origin/feature/login has moved since your last fetch
  → Rejects push: "stale info; rejecting to avoid race condition"
  → You must git fetch first, see what's there, then decide
```

When Force Push Is Acceptable

```
✅ Your personal feature branch that only you work on
✅ Cleaning up WIP commits before opening a PR
✅ After interactive rebase of a feature branch
```

```
❌ NEVER on main, develop, release, or any shared branch
❌ Never in production deployment pipelines
```

Short Interview Answer

> "`git reset` moves the branch pointer and optionally clears staging and working directory — it rewrites history and is only safe on local or unshared commits. `git revert` creates a new commit that inverts the changes, preserving history — always safe on shared branches. For force push, I always use `--force-with-lease` instead of `--force` because it checks that no one else has pushed to the remote since my last fetch, preventing accidental overwrites of teammates' work."

⚠ Gotchas

- `git reset --hard` with no arguments resets to HEAD — useful for discarding all uncommitted changes.
- Rebased commits have different SHAs even if the diff is identical. CI systems referencing old SHAs will fail.
- `--force-with-lease` can be defeated if you ran `git fetch` before the push — it updates your remote-tracking ref, so it no longer detects the conflict. Some teams use `--force-if-includes` (Git 2.30+) for additional safety.

I3. Advanced Merging and Rebasing

Merge Strategies — `ours`, `theirs`, `octopus`, `recursive`, `ort`

What Merge Strategies Are

Git uses a strategy to determine HOW to combine two or more branches. Most merges use the default automatically, but you can override for specific scenarios.

`recursive` (default before Git 2.33)

Handles the **criss-cross merge** problem. When two branches have multiple common ancestors, `recursive` creates a virtual merge base by merging the common ancestors first, then uses that as the base for the final merge.

```
git merge -s recursive feature/login
```

`ort` (default since Git 2.33)

"Ostensibly Recursive's Twin" — rewritten from scratch, faster and more correct than `recursive` especially for complex rename detection and large repos.

```
git merge -s ort feature/login
```

`ours`

Merges by always taking the current branch's version for every conflict. The other branch is effectively discarded but recorded as merged.

```
# Merge feature/old-ui but completely keep current branch's files
git merge -s ours feature/old-ui
# Creates a merge commit but all file content is from current branch
# Useful for: marking a branch as "merged" without taking its changes
# Use case: deprecating a branch while keeping its history visible
```

⚠ **`-s ours` (strategy) vs `-X ours` (strategy option) are DIFFERENT:**

```
# -s ours: entire merge uses current branch (no per-file conflict
resolution)
git merge -s ours feature/branch

# -X ours: use recursive/ort strategy but resolve ALL conflicts favoring
ours
git merge -X ours feature/branch

# -X theirs: resolve ALL conflicts favoring theirs
git merge -X theirs feature/branch
```

`octopus`

Merges **more than two branches** at once. Used for integrating multiple feature branches simultaneously.

```
git merge feature/login feature/payments feature/notifications
# Octopus strategy used automatically for 3+ branches
# Fails if there are any conflicts — requires clean merges only
```

`ours` as a Strategy Option (per-file)

```
# During a merge conflict, take your version of a specific file
git checkout --ours src/config.js
git add src/config.js

# Take incoming version of a specific file
git checkout --theirs src/config.js
git add src/config.js
```

⚠ Rebase vs Merge — Philosophical and Practical Differences

The Core Tradeoff

```
Merge:  preserves the true history of what happened
Rebase: creates a cleaner, linear history that is easier to read
```

Visual Comparison

```
Starting state:
  main:    C1 → C2 → C3
                ↑
  feature:      F1 → F2

After git merge main into feature:
  feature: F1 → F2 → M (merge commit, two parents: F2 and C3)
  True history: shows that development happened in parallel

After git rebase main on feature:
  feature: F1' → F2' (replayed on top of C3)
  False history: appears as if feature was written after C3
```

When to Use Each

Scenario	Use
Keeping a feature branch up to date with main	`rebase`
Integrating a completed feature into main	`merge --no-ff`
Main is a public, shared branch	`merge` always
Personal feature branch cleanup	`rebase -i`
Open source contributions (clean history)	`rebase`
Regulated environments needing audit trail	`merge`
Bisecting bugs (linear history helps)	`rebase` preferred

The Golden Rule of Rebase

> **Never rebase commits that have been pushed to a shared branch.**

When you rebase, old commits get new SHAs. Anyone who based work on the old SHAs now has diverged history. This causes the "rebasing catastrophe" — everyone has to run `git pull --rebase` or `git reset --hard` to recover.

Short Interview Answer

> "Merge preserves true history — you can see exactly when branches diverged and converged. Rebase rewrites history to be linear — easier to read and bisect, but loses the parallel development context. I use rebase to keep my feature branch current with main and to clean up WIP commits before opening a PR. I use merge to integrate completed features because it creates a clear record that a feature was developed separately. The rule I never break: never rebase commits already on a shared branch."

`git cherry-pick` — Use Cases, Ranges, and Risks

What it is

Cherry-pick takes a specific commit from anywhere in the history and **applies its changes** to the current branch as a new commit with a new SHA.

```
# Cherry-pick a single commit
git cherry-pick abc1234

# Cherry-pick a range (from older to newer, exclusive start)
git cherry-pick abc1234..def5678

# Cherry-pick a range (inclusive start)
git cherry-pick abc1234^..def5678

# Cherry-pick without committing (stage only)
git cherry-pick --no-commit abc1234

# Cherry-pick and preserve original author
git cherry-pick -x abc1234
# Adds "Cherry-picked from commit abc1234" to message

# Abort cherry-pick in progress
git cherry-pick --abort

# Continue after resolving conflict
git cherry-pick --continue
```

Real-World Use Cases

```
1. Hotfix backporting:
   Bug fixed in main → cherry-pick to release/2.1.x branch

2. Selective feature inclusion:
   A feature branch has 10 commits, only 3 are ready → cherry-pick those 3

3. Cross-repo porting:
   Apply a specific fix from one repo to another
```

```
4. Recovering a commit dropped by rebase:
   Rebase went wrong, found the commit in reflog → cherry-pick it back
```

⚠ Risks

- Creates a **duplicate commit** (same changes, different SHA). If the original is later merged, you get a conflict or double application.
- Cherry-picking a merge commit is complex and requires `-m` flag to specify which parent's diff to use.
- Cherry-picking across long histories can miss context that the original commit depended on.

`git rerere` — Reuse Recorded Resolution

What it is

`rerere` = **Re**use **Re**corded **Re**solution. When you resolve a merge conflict, Git can remember how you resolved it. Next time the same conflict appears, it automatically applies your resolution.

```
# Enable rerere globally
git config --global rerere.enabled true

# Enable rerere for current repo only
git config rerere.enabled true

# See recorded resolutions
ls .git/rr-cache/

# Apply rerere resolutions manually
git rerere

# Clear rerere cache for a specific conflict
git rerere forget path/to/file
```

Why It's Valuable

In long-lived feature branches that regularly rebase onto main, you often hit the same conflicts repeatedly (someone keeps modifying the same config file). With `rerere`, you resolve it once and Git handles it automatically in future rebases.

Real-World CI/CD Use

Teams using **merge queues** where PRs are rebased and retested automatically benefit from rerere — without it, every retest cycle might hit the same conflicts manually.

`git merge --no-ff` and Why It Matters for History

```
# Default merge (fast-forward if possible, merge commit otherwise)
git merge feature/login

# Force a merge commit even if fast-forward is possible
git merge --no-ff feature/login

# Force fast-forward only (fails if not possible)
git merge --ff-only feature/login
```

Why `--no-ff` Matters

```
With fast-forward (default when possible):
  main: C1 → C2 → F1 → F2   (feature work looks like it was on main)
  No record that feature/login ever existed as a separate branch

With --no-ff:
  main: C1 → C2 → M          (M is a merge commit)
                 ↗
  feature:   F1 → F2
  Git history shows the feature branch clearly
```

`--no-ff` is preferred in GitFlow because it preserves the branch structure in history. `git log --graph` clearly shows every feature, making it easy to revert an entire feature by reverting the merge commit.

I4. Stashing and Worktrees

`git stash` — Push, Pop, Apply, List, Drop, Clear

What it is (Simple Terms)

Stash temporarily shelves changes in the working directory and staging area so you can switch context (e.g., fix a hotfix) and come back later.

```
# Stash current changes (tracked files only)
git stash
git stash push           # explicit form
git stash push -m "WIP: OAuth integration work"   # named stash

# Stash including untracked files
git stash -u
git stash push --include-untracked

# Stash including ignored files too
git stash -a
```

```
git stash push --all

# List all stashes
git stash list
# stash@{0}: WIP on feature/login: a3f9d2c feat: add login form
# stash@{1}: On main: 8ab686e fix: button alignment

# Apply most recent stash (keeps it in stash list)
git stash apply

# Apply and remove from stash list
git stash pop

# Apply a specific stash
git stash apply stash@{2}
git stash pop stash@{1}

# Show what's in a stash (diff)
git stash show stash@{0}
git stash show -p stash@{0}    # full diff

# Drop (delete) a specific stash
git stash drop stash@{1}

# Clear all stashes ⚠ Irreversible
git stash clear

# Create a branch from a stash (best way to resume long-stashed work)
git stash branch feature/resumed stash@{0}
# Creates branch, checks it out, applies stash, drops it
```

How Stash Works Internally

A stash creates two commits in a special area:

1. One commit for the staged changes (index state)
2. One commit for the unstaged working directory changes Both are stored as refs under `refs/stash` and linked to your current HEAD.

⚠ Gotchas

- `git stash pop` = `git stash apply` + `git stash drop`. If apply causes a conflict, the stash is NOT dropped — the conflict must be resolved first, then manually drop.
- Stashes are per-repository, not per-branch. You can apply a stash created on one branch to a different branch (useful but can cause conflicts).
- `git stash` doesn't stash untracked files by default. You MUST use `-u`.
- `git stash clear` is permanent. There is no undo. Reflog doesn't help here.

Named Stashes and Stash Branching

```
# Always name your stashes for long-lived work
git stash push -m "WIP: payment gateway integration, 60% done"

# Resume work safely via branch (avoids conflicts from diverged branch)
git stash branch payment-resumed stash@{0}
```

`git worktree` — Multiple Working Trees from One Repo

What it is

`git worktree` lets you check out **multiple branches simultaneously** from the same repository into different directories — without cloning.

```
# Add a worktree for a branch
git worktree add ../hotfix-branch hotfix/payment-bug

# Add a worktree creating a new branch
git worktree add -b feature/new-ui ../new-ui-work main

# List all worktrees
git worktree list
# /home/jane/main-repo        a3f9d2c [main]
# /home/jane/hotfix-branch    b7e1234 [hotfix/payment-bug]
# /home/jane/new-ui-work      c4d5678 [feature/new-ui]

# Remove a worktree (must not be in use)
git worktree remove ../hotfix-branch

# Prune stale worktree references
git worktree prune
```

Why Use Worktrees Over Stash or Clone

Scenario	Stash	New Clone	Worktree
Speed	Fast	Slow (re-download)	Fast (shared objects)
Disk space	Minimal	Full copy	Minimal (shared `.git`)
Isolated builds	No	Yes	Yes
Multiple branches simultaneously	No	Possible	Yes

Real-World Use Cases in CI/CD and SRE

```
1. Hotfix while feature work is in progress:
   Main worktree: feature/payments (WIP, 200 changed files)
```

```
   New worktree: hotfix/critical-security-fix
   → Fix the hotfix, deploy, return to feature — no stash needed

2. Parallel builds:
   Build main and develop simultaneously from the same repo
   → No two git clones taking double disk space

3. Code review:
   Review a colleague's branch in a second worktree
   while your own work continues untouched in the main worktree

4. CI agent efficiency:
   Single bare repo clone shared across multiple build jobs
   Each job checks out its branch into a separate worktree
```

Short Interview Answer

> "Git stash temporarily shelves uncommitted changes so you can switch context, then restore them later. The key flags are `-u` for untracked files and `-m` for naming stashes. `git worktree` is a step further — it gives you multiple working directories from a single repo, each on a different branch simultaneously. I use stash for quick context switches and worktrees for longer parallel work like fixing a hotfix while a feature is mid-development, since worktrees share the object store and don't require a full re-clone."

I5. Branching Strategies ⚠

> One of the most discussed topics in DevOps interviews. Interviewers want to know not just what each strategy is, but when and why you'd choose it.

GitFlow — Feature, Develop, Release, Hotfix, Main

What it is

GitFlow is a strict branching model with well-defined roles for each branch. Designed for **scheduled, versioned releases**.

Branch Structure

```
main (or master)    → production-ready code, tagged with version numbers
develop             → integration branch, latest delivered development
feature/*           → new features branched from develop
release/*           → release preparation branched from develop
hotfix/*            → urgent production fixes branched from main
```

The GitFlow Lifecycle

```
1. Feature development:
   git checkout -b feature/oauth develop
   # ... work ...
   git checkout develop
   git merge --no-ff feature/oauth
   git branch -d feature/oauth

2. Release preparation:
   git checkout -b release/2.1.0 develop
   # bump version, fix release bugs only
   git checkout main
   git merge --no-ff release/2.1.0
   git tag -a v2.1.0
   git checkout develop
   git merge --no-ff release/2.1.0    # ← don't forget this!
   git branch -d release/2.1.0

3. Hotfix:
   git checkout -b hotfix/2.1.1 main
   # fix the bug
   git checkout main
   git merge --no-ff hotfix/2.1.1
   git tag -a v2.1.1
   git checkout develop
   git merge --no-ff hotfix/2.1.1     # ← don't forget this!
   git branch -d hotfix/2.1.1
```

When GitFlow Works Well

- Mobile apps with controlled App Store releases
- Enterprise software with versioned releases (v2.1, v2.2)
- Products where multiple versions are supported simultaneously

GitFlow Problems

- Complex merge flows — easy to forget to merge hotfix back to develop
- Long-lived branches → big merge conflicts
- Too heavy for teams doing continuous deployment
- The `develop` branch can become a conflict magnet

GitHub Flow — Simplicity Optimized for Continuous Delivery

What it is

GitHub Flow has only one long-lived branch (`main`) and short-lived feature branches. Every change goes through a PR, and main is always deployable.

The Workflow

```
main                      → always deployable, always production-ready
feature/*, fix/*, etc.    → short-lived, branch from main, PR back to main
```

```
# Full GitHub Flow cycle
git checkout -b feature/add-oauth main
# ... develop ...
git push -u origin feature/add-oauth
# Open PR → code review → CI passes → merge to main → auto-deploy
git branch -d feature/add-oauth
git push origin --delete feature/add-oauth
```

Why It's Powerful

- Simple — one rule: main is always deployable
- Fast — short-lived branches mean small PRs and less conflict
- Aligns with CI/CD — merge to main = deploy
- Scales to large teams easily

Limitation

No built-in mechanism for managing multiple release versions. If you need to patch v1.x while v2.x is in development, GitHub Flow doesn't prescribe how.

Trunk-Based Development — Why Google, Netflix, and Meta Use It

What it is

Everyone commits to a single trunk (`main`) directly or through very short-lived branches (< 1 day). No long-lived feature branches. Feature flags control what's visible to users.

```
main ← everyone commits here (or through same-day PRs)
     ← feature flags hide incomplete work
     ← CI/CD deploys continuously
```

Why Industry Leaders Use It

```
Google: Single repo (monorepo), millions of commits/year to trunk
Netflix: Deploys to production hundreds of times/day
Meta: trunk-based with feature flags for billions of users
```

The key insight: Long-lived branches are where complexity, conflicts, and integration problems hide. The longer a branch lives, the worse the merge. Trunk-based development forces **continuous integration** in the

literal sense.

Making TBD Work

```
# Short-lived branch approach (< 1 day)
git checkout -b feat/payment-button main
# ... implement small increment ...
git push origin feat/payment-button
# PR reviewed and merged same day → branch deleted

# Feature flags for incomplete work
if (featureFlags.isEnabled('new-payment-ui', user)) {
  renderNewUI();
} else {
  renderOldUI();
}
```

TBD Requirements

- Excellent test coverage (you're always deploying main)
- Feature flags infrastructure
- Strong code review culture (everyone reviews quickly)
- Robust CI/CD pipeline (quality gate before merge)
- Monorepo or well-bounded services

Release Branching vs Feature Flags

Release Branches

```
main → development
release/2.1.x → stabilization and patches for 2.1 series
release/2.0.x → long-term support patches
```

Pro: Isolates stability work. Clear code freeze. **Con:** Long-lived branches, divergence, backporting complexity.

Feature Flags

```
main → always deployable
Code contains both old and new paths, controlled by flags
```

```
if feature_flags.get('new_checkout_flow', user_id=user.id):
    return new_checkout()
```

```
else:
    return legacy_checkout()
```

Pro: Decouple deployment from release. A/B testing. Instant rollback (flip the flag). **Con:** Technical debt accumulation if flags aren't cleaned up. Testing complexity (must test both paths).

Environment Branches — Anti-Pattern or Valid?

What Environment Branches Are

```
main → production
staging → staging environment
develop → development environment
```

Changes flow: feature → develop → staging → main

Why This Is Mostly an Anti-Pattern

- Changes don't flow cleanly — manual cherry-picks or complex promotion flows
- Branches diverge — what's in staging isn't what's in production
- "It works in staging" syndrome — the branches are genuinely different
- Promotes merge hell rather than automation

The Better Alternative

Use **the same branch (main)** with **deployment pipelines** controlling which environment is targeted:

```
main → CI pipeline deploys to: dev → staging → prod
         (same code, different config, controlled by pipeline)
```

Choosing the Right Strategy

Team Size	Release Model	Recommended Strategy
1-5 developers	Continuous deploy	GitHub Flow or TBD
5-20 developers	Versioned releases	GitFlow or GitHub Flow
20-100 developers	Continuous deploy	TBD with feature flags
100+ developers	Continuous deploy	TBD mandatory
Multiple version support	Versioned	GitFlow or release branches

Short Interview Answer

> "GitFlow suits teams with scheduled versioned releases — it provides clear structure for features, releases, and hotfixes but becomes unwieldy for continuous deployment. GitHub Flow is simpler — just main plus short-lived feature branches, merge to deploy. Trunk-Based Development is what Google, Netflix, and Meta use — everyone works on main continuously with feature flags hiding incomplete work. For a team doing multiple deploys per day, TBD is the answer because long-lived branches are where integration pain hides. I choose based on team size, release cadence, and whether the team has the discipline and tooling for TBD."

I6. Git Hooks

What Hooks Are

Git hooks are **scripts** that run automatically at specific points in the Git lifecycle. They live in `.git/hooks/` and can be written in any language (bash, Python, Ruby, Node.js).

```
ls .git/hooks/
# applypatch-msg.sample   pre-applypatch.sample
# commit-msg.sample       pre-commit.sample
# post-commit.sample      pre-push.sample
# post-merge.sample       pre-rebase.sample
# post-receive.sample     update.sample
# pre-receive.sample      prepare-commit-msg.sample

# Activate a hook: remove .sample extension and make executable
cp .git/hooks/pre-commit.sample .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
```

Client-Side Hooks

`pre-commit`

Runs before the commit message editor opens. Exit non-zero to abort the commit.

```
#!/bin/bash
# .git/hooks/pre-commit

# Run linter
npm run lint
if [ $? -ne 0 ]; then
  echo "❌ Lint failed. Fix errors before committing."
  exit 1
fi

# Run tests
npm test
```

```
if [ $? -ne 0 ]; then
  echo "❌ Tests failed. Fix tests before committing."
  exit 1
fi

# Check for debug code
if git diff --cached --name-only | xargs grep -l
"console.log\|debugger\|TODO" 2>/dev/null; then
  echo "⚠  Warning: Debug code or TODOs found in staged files."
  # exit 1  # uncomment to block
fi

echo "✅ Pre-commit checks passed."
exit 0
```

commit-msg

Receives the commit message file path as `$1`. Used to enforce commit message format.

```
#!/bin/bash
# .git/hooks/commit-msg

COMMIT_MSG=$(cat "$1")
PATTERN="^(feat|fix|docs|style|refactor|test|chore|perf|ci|build|revert)(\
(.+\))?: .{1,72}$"

if ! echo "$COMMIT_MSG" | grep -qE "$PATTERN"; then
  echo "❌ Invalid commit message format."
  echo "   Required: type(scope): description"
  echo "   Example:  feat(auth): add OAuth2 login"
  echo "   Types:
feat|fix|docs|style|refactor|test|chore|perf|ci|build|revert"
  exit 1
fi

echo "✅ Commit message format valid."
exit 0
```

prepare-commit-msg

Runs before the commit message editor, after the default message is set. Useful for auto-populating messages.

```
#!/bin/bash
# Auto-prepend branch ticket number to commit message
# Branch: feature/JIRA-123-oauth-login → prepends [JIRA-123]

BRANCH=$(git symbolic-ref --short HEAD)
TICKET=$(echo "$BRANCH" | grep -oE '[A-Z]+-[0-9]+')
```

```
if [ -n "$TICKET" ]; then
  sed -i "1s/^/[$TICKET] /" "$1"
fi
```

pre-push

Runs before `git push`. Receives remote name and URL via stdin.

```
#!/bin/bash
# .git/hooks/pre-push

# Run full test suite before push
echo "Running tests before push..."
npm test
if [ $? -ne 0 ]; then
  echo "❌ Tests failed. Push aborted."
  exit 1
fi

# Prevent direct push to main
REMOTE="$1"
BRANCH=$(git symbolic-ref --short HEAD)
if [ "$BRANCH" = "main" ] && [ "$REMOTE" = "origin" ]; then
  echo "❌ Direct push to main is not allowed. Use a PR."
  exit 1
fi

exit 0
```

Server-Side Hooks

pre-receive

Runs before any refs are updated on the server. Receives list of old SHA, new SHA, and ref name on stdin. Most powerful for enforcement.

```
#!/bin/bash
# server-side: .git/hooks/pre-receive

while read oldsha newsha refname; do
  branch=$(echo "$refname" | sed 's|refs/heads/||')

  # Block direct push to main
  if [ "$branch" = "main" ]; then
    echo "❌ Direct push to main is not allowed. Use pull requests."
    exit 1
  fi
```

```
  # Check commit messages in the push
  git log "$oldsha..$newsha" --format="%H %s" | while read sha msg; do
    if ! echo "$msg" | grep -qE
"^(feat|fix|docs|style|refactor|test|chore)"; then
      echo "❌ Commit $sha has invalid message: $msg"
      exit 1
    fi
  done
done
exit 0
```

`update`

Similar to `pre-receive` but runs once per ref being updated. Exit non-zero to reject that specific ref.

`post-receive`

Runs after all refs are updated. Used for notifications, deployments, CI triggers.

```
#!/bin/bash
# server-side: .git/hooks/post-receive
# Trigger CI/CD pipeline or send Slack notification

while read oldsha newsha refname; do
  branch=$(echo "$refname" | sed 's|refs/heads/||')
  echo "Notifying CI for branch: $branch"
  curl -X POST "https://ci.company.com/api/trigger" \
    -d "{\"branch\": \"$branch\", \"sha\": \"$newsha\"}"
done
```

Managing Hooks at Scale — Husky and pre-commit Framework

The Problem

`.git/hooks/` is NOT committed to the repository. Each developer has to manually install hooks. On a team of 50, this breaks down.

Husky (JavaScript/Node.js projects)

```
# Install
npm install --save-dev husky
npx husky init

# .husky/pre-commit
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"
npx lint-staged
```

```
# .husky/commit-msg
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"
npx commitlint --edit "$1"

# package.json
{
  "scripts": {
    "prepare": "husky install"
  },
  "lint-staged": {
    "*.js": ["eslint --fix", "prettier --write"],
    "*.ts": ["tsc --noEmit", "eslint --fix"]
  }
}
```

pre-commit Framework (Python/polyglot)

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.4.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-json
      - id: detect-private-key           # ← catches accidental secret
commits

  - repo: https://github.com/psf/black
    rev: 23.1.0
    hooks:
      - id: black

  - repo: https://github.com/PyCQA/flake8
    rev: 6.0.0
    hooks:
      - id: flake8
```

```
# Install
pip install pre-commit
pre-commit install         # installs to .git/hooks/pre-commit

# Run manually on all files
pre-commit run --all-files

# Update hook versions
pre-commit autoupdate
```

Bypassing Hooks (`--no-verify`) and the Risks

```
# Skip ALL client-side hooks
git commit --no-verify -m "emergency: bypass hooks for hotfix"
git push --no-verify

# When is this acceptable?
# → Genuine emergency hotfix where every second counts
# → Hook is broken/misconfigured and blocking valid work
# → CI will catch what the hook would have caught
```

⚠ The Risk

`--no-verify` is routinely abused. Developers learn to use it to escape slow test hooks. Once it's normalized, the safety net degrades. Server-side hooks (`pre-receive`) cannot be bypassed by clients — always enforce critical rules server-side.

Short Interview Answer

> "Git hooks are scripts that run at lifecycle events — pre-commit for linting and tests, commit-msg for enforcing commit format, pre-push to run the full suite before pushing, and server-side pre-receive to enforce team policies that can't be bypassed. The challenge is distribution — `.git/hooks/` isn't committed. I solve this with Husky for Node.js projects or the pre-commit framework for polyglot repos, so hooks are checked in and installed via `npm install` or `pre-commit install`. Critical rules always go server-side because `--no-verify` can bypass client hooks."

I7. Submodules and Subtrees

`git submodule` — What It Is, How `.gitmodules` Works

What it is

A submodule embeds another Git repository inside your repository at a specific commit. The parent repo stores a pointer to the exact commit of the submodule — not the content itself.

```
# Add a submodule
git submodule add git@github.com:org/shared-components.git libs/shared

# What gets created:
# .gitmodules file (tracked)
# libs/shared/ directory (tracked as a gitlink, not full content)

cat .gitmodules
```

```
# [submodule "libs/shared"]
#   path = libs/shared
#   url = git@github.com:org/shared-components.git
#   branch = main  # optional: track a specific branch

# The parent repo stores a "gitlink" — just the submodule SHA
git ls-files -s libs/shared
# 160000 a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d 0 libs/shared
#  ^mode 160000 = gitlink (submodule)
```

Cloning Repos with Submodules

```
# Clone and initialize submodules in one step
git clone --recurse-submodules git@github.com:org/main-repo.git

# Initialize submodules after a plain clone
git submodule init
git submodule update

# Both in one command
git submodule update --init --recursive   # --recursive for nested
submodules
```

Working with Submodules

```
# Update submodule to latest commit on its tracked branch
git submodule update --remote libs/shared

# Update all submodules to their recorded commits (after git pull)
git submodule update --init --recursive

# Enter submodule and work on it
cd libs/shared
git checkout main
# make changes, commit, push
cd ..

# Record the new submodule commit in the parent
git add libs/shared
git commit -m "chore: update shared-components to v2.3.0"

# Run a command in all submodules
git submodule foreach 'git pull origin main'
```

⚠ Submodule Pitfalls in CI/CD Pipelines

```
Pitfall 1: git clone doesn't fetch submodules by default
Fix: Always use --recurse-submodules or git submodule update --init

Pitfall 2: Submodule is on a detached HEAD by default
Fix: cd into submodule, git checkout main, work, commit, push

Pitfall 3: Submodule commit pointer updated but not committed in parent
Fix: After updating submodule, git add <submodule-path> + commit

Pitfall 4: Shallow clone misses submodule objects
Fix: git clone --recurse-submodules --shallow-submodules --depth=1

Pitfall 5: SSH URLs in .gitmodules fail in CI (no SSH keys)
Fix: git config --global url."https://".insteadOf "git@"
     or use HTTPS URLs in .gitmodules directly

Pitfall 6: Nested submodules compound all of the above
Fix: git submodule update --init --recursive (--recursive covers nesting)
```

CI Pipeline Fix for Submodules

```yaml
# GitHub Actions example
- name: Checkout with submodules
  uses: actions/checkout@v3
  with:
    submodules: recursive
    token: ${{ secrets.GITHUB_TOKEN }}
```

`git subtree` — Difference from Submodules

What it is

`git subtree` merges another repository into a **subdirectory** of your repo, as actual committed files — not a pointer.

```
# Add a subtree
git subtree add --prefix=libs/shared git@github.com:org/shared-
components.git main --squash

# Pull updates from the remote
git subtree pull --prefix=libs/shared git@github.com:org/shared-
components.git main --squash

# Push changes from subdirectory back to the remote
git subtree push --prefix=libs/shared git@github.com:org/shared-
components.git main
```

Submodule vs Subtree Comparison

Feature	Submodule	Subtree
Storage	Pointer to commit in parent	Actual files in parent repo
Plain `git clone`	❌ Must initialize separately	✅ Works out of the box
Contributor complexity	High (two repos to manage)	Low (one repo, normal git)
History	Separate history per repo	Merged into parent history
Updating	`git submodule update`	`git subtree pull`
Contributing back	Commit inside submodule dir	`git subtree push`
CI complexity	High	Low
Large binary content	Manageable	Bloats parent repo

When to Use Which

```
Use submodules when:
  → The dependency has its own independent release cycle
  → Multiple parent repos share the same dependency
  → You need precise version pinning (specific commit)
  → The team is comfortable with the submodule workflow

Use subtrees when:
  → Simplicity is paramount (contributors shouldn't need to know about it)
  → The dependency is infrequently updated
  → You want the dependency visible in git log naturally

Use neither (use a package manager instead) when:
  → The dependency is a library (use npm, pip, Maven, etc.)
  → You don't need to modify the dependency
```

Short Interview Answer

> "Submodules embed another repo as a pointer — the parent stores the exact SHA of the submodule, not its files. This enables precise versioning but adds complexity: plain clones don't fetch submodule content, contributors must know to run `git submodule update`, and CI pipelines must handle them explicitly. Subtrees merge the other repo's files directly into a subdirectory — simpler for contributors but bloats history. For most teams I recommend package managers over either, and if code sharing is needed across repos, subtrees for simplicity or submodules when strict version pinning across multiple consumers is required."

I8. Tagging and Versioning

Lightweight vs Annotated Tags — Internals and Differences

Lightweight Tags (Recap + Internals)

A lightweight tag is simply a file in `.git/refs/tags/` containing a commit SHA. No object created.

```
git tag v2.0.0-beta           # creates .git/refs/tags/v2.0.0-beta

cat .git/refs/tags/v2.0.0-beta
# a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d  ← points directly to commit
```

Annotated Tags (Internals)

An annotated tag creates a **tag object** in `.git/objects/`. The tag ref points to this object, which points to the commit.

```
git tag -a v2.0.0 -m "Release 2.0.0 - OAuth2 integration"

cat .git/refs/tags/v2.0.0
# b7e12340f5a8b9c6d1e2f3a4b5c6d7e8f9a0b1c2   ← points to TAG OBJECT

git cat-file -t b7e1234   # type: tag
git cat-file -p b7e1234
# object a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d   ← points to commit
# type   commit
# tag    v2.0.0
# tagger Jane Smith <jane@company.com> 1700000000 +0530
#
# Release 2.0.0 - OAuth2 integration
```

Semantic Versioning (semver) with Git Tags

```
MAJOR.MINOR.PATCH[-prerelease][+build]

v2.1.0          → stable release
v2.1.1          → patch (backwards-compatible bug fix)
v2.2.0          → minor (backwards-compatible new feature)
v3.0.0          → major (breaking change)
v2.1.0-alpha.1  → pre-release alpha
v2.1.0-beta.2   → pre-release beta
v2.1.0-rc.1     → release candidate
```

```
# Tagging a release following semver
git tag -a v2.1.0 -m "Release v2.1.0
```

```
Features:
- OAuth2 Google provider
- OAuth2 GitHub provider

Bug fixes:
- Session timeout handling
- Token refresh edge cases"

git push origin v2.1.0
```

`git describe` for Dynamic Version Generation

What it is

`git describe` generates a human-readable version string based on the nearest tag.

```
# Output: <nearest-tag>-<commits-since-tag>-g<short-sha>
git describe
# v2.1.0-14-ga3f9d2c

# On an exact tag:
git describe
# v2.1.0

# Only use annotated tags (default behavior)
git describe

# Include lightweight tags
git describe --tags

# Always show long format (even on exact tag)
git describe --long
# v2.1.0-0-ga3f9d2c

# Fail if no tag found
git describe --exact-match HEAD  # exits non-zero if not on a tag
```

Using `git describe` in CI for Version Strings

```
#!/bin/bash
# Generate version for Docker image or artifact
VERSION=$(git describe --tags --always --dirty)
# Examples:
# v2.1.0           → exact tag (clean build)
# v2.1.0-14-ga3f9d  → 14 commits past v2.1.0
# v2.1.0-14-ga3f9d-dirty → uncommitted changes present
```

```
echo "Building version: $VERSION"
docker build -t myapp:$VERSION .
docker push myapp:$VERSION
```

Automated Tagging in CI Pipelines

Strategy 1 — Manual Tag Triggers Deployment

```yaml
# GitHub Actions: deploy only on tag push
on:
  push:
    tags:
      - 'v*.*.*'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Get version
        run: echo "VERSION=${GITHUB_REF#refs/tags/}" >> $GITHUB_ENV
      - name: Deploy
        run: ./deploy.sh $VERSION
```

Strategy 2 — Auto-Tag on Merge to Main

```bash
#!/bin/bash
# CI script: auto-increment patch version and tag

# Get latest tag
LATEST_TAG=$(git describe --tags --abbrev=0 2>/dev/null || echo "v0.0.0")

# Parse version
IFS='.' read -ra PARTS <<< "${LATEST_TAG#v}"
MAJOR=${PARTS[0]}
MINOR=${PARTS[1]}
PATCH=${PARTS[2]}

# Increment patch
NEW_PATCH=$((PATCH + 1))
NEW_TAG="v${MAJOR}.${MINOR}.${NEW_PATCH}"

echo "Tagging as $NEW_TAG"
git tag -a "$NEW_TAG" -m "Automated release $NEW_TAG"
git push origin "$NEW_TAG"
```

Strategy 3 — Conventional Commits Drive Versioning

Tools like `semantic-release` read commit messages to decide version bumps:

```
feat: → minor version bump (v2.1.0 → v2.2.0)
fix:  → patch version bump (v2.1.0 → v2.1.1)
BREAKING CHANGE: → major version bump (v2.1.0 → v3.0.0)
```

```
# .releaserc.json
{
  "branches": ["main"],
  "plugins": [
    "@semantic-release/commit-analyzer",
    "@semantic-release/release-notes-generator",
    "@semantic-release/changelog",
    "@semantic-release/git",
    "@semantic-release/github"
  ]
}
```

Tag-Triggered Deployment Pipelines

```
Developer creates and pushes tag → CI/CD detects tag push → runs
deployment pipeline

Benefits:
  → Humans control exactly when a deployment happens
  → The tag IS the deployment artifact identifier
  → Easy rollback: deploy the previous tag
  → Clear audit trail in Git history
```

```
# Jenkins pipeline triggered by tag
pipeline {
  triggers {
    // Trigger only on version tags
    githubPush()
  }
  stages {
    stage('Detect Tag') {
      steps {
        script {
          def tag = sh(script: 'git describe --exact-match --tags HEAD',
returnStdout: true).trim()
          if (!tag.startsWith('v')) {
```

```
            currentBuild.result = 'ABORTED'
            error("Not a version tag — skipping deployment")
          }
          env.VERSION = tag
        }
      }
    }
    stage('Deploy') {
      steps {
        sh "./deploy.sh ${env.VERSION}"
      }
    }
  }
}
```

⚠ Pushing Tags to Remotes

```
# Tags are NEVER pushed by git push by default

# Push a single tag
git push origin v2.1.0

# Push all tags
git push --tags

# Push all tags along with the current branch
git push --follow-tags    # ← preferred in CI (only pushes annotated tags)

# Fetch tags from remote (also NOT fetched by git fetch by default in some
configs)
git fetch --tags
git fetch origin --tags
```

Why This Matters in CI/CD

```
Problem: CI runs git clone --depth=1 → no tags fetched
         git describe fails → version string is just a short SHA
         Docker image tagged as "ga3f9d2c" instead of "v2.1.0-5-ga3f9d2c"

Fix:
  git fetch --tags --unshallow
  # or
  git fetch --tags origin
  # or in GitHub Actions:
  - uses: actions/checkout@v3
    with:
      fetch-depth: 0   # 0 = full history + all tags
```

Short Interview Answer

> "Tags mark specific commits in history — lightweight tags are just SHA pointers while annotated tags are full objects with metadata and can be GPG-signed. For releases I always use annotated tags with semver. `git describe` generates version strings from the nearest tag which I use in CI to tag Docker images and artifacts. Tags must be pushed explicitly — `git push --follow-tags` is my go-to in CI. The most common pitfall is shallow clones in CI not fetching tags, which breaks `git describe` — fix with `fetch-depth: 0` or `git fetch --tags`."

🔗 How I1–I8 Connect to Each Other and to B1–B4

```
Git Object Model (I1)
  ↓ explains why
Rewriting History (I2) changes SHAs → force push needed → --force-with-
lease
  ↓ directly affects
Advanced Merging (I3) → merge strategies resolve conflicts created by
diverged histories
  ↓ connects to
Branching Strategies (I5) → which strategy determines HOW branches diverge
and merge
  ↓ enforced by
Git Hooks (I6) → server-side hooks enforce branching policies
automatically
  ↓ applied in
Tags and Versioning (I8) → hooks auto-tag on merge to main, triggering
deployments

Stash / Worktrees (I4) → practical tools for managing context switches in
any workflow
Submodules / Subtrees (I7) → repo organization decisions that affect CI/CD
clone strategies

All of I1–I8 connect to:
  Advanced: Large Repo Strategies (A3) → shallow clones, sparse checkout
  Advanced: Git in CI/CD (A5)          → all the gotchas surface in
pipelines
  Advanced: Troubleshooting (A7)       → reflog, fsck, recovery using
object model knowledge
```

📋 Quick Reference — Most Frequently Asked Intermediate Questions

Question	One-Line Answer

Question	One-Line Answer
Rebase vs merge	Rebase = linear history, rewrites SHA; Merge = preserves history, creates merge commit
`git reset --soft` vs `--hard`	soft = undo commit, keep staged; hard = undo commit AND discard changes
`git revert` vs `git reset`	revert = safe, creates undo commit; reset = rewrites history, unsafe on shared branches
`--force` vs `--force-with-lease`	force-with-lease fails if remote has new commits you haven't fetched
What is HEAD	Pointer to current branch (or commit in detached HEAD state)
What is ORIG_HEAD	Saved HEAD before dangerous operations — used for undo
How does `git stash` work internally	Creates two commits (index state + working dir state) under refs/stash
Submodule vs subtree	Submodule = pointer to other repo; Subtree = files merged into parent repo
Why use annotated tags	Full object with metadata, GPG-signable, shown by `git describe` by default
Trunk-based vs GitFlow	TBD = everyone on main + feature flags; GitFlow = multiple long-lived branches for releases
What is `git rerere`	Records conflict resolutions, auto-applies them on future identical conflicts
When is force push acceptable	Only on personal feature branches never shared with others

Next: Advanced section A1–A8 covering internals deep dive, large repo strategies, Git security, system design questions, and recovery scenarios.